

Visual Reasoning Question Engine

Architecture & Synthetic Dataset Plan — Paper Folding/Punching, Mirror & Rotation Reasoning

Prepared for aptitude / pilot-selection style reasoning question generation (SSC, UPSC, DRDO, CAT format)

1. Summary

You don't need a document retrieval system (RAG) to generate new folding, punching, mirror, and rotation reasoning questions — you need a procedural geometry engine that constructs each question from first principles, so the correct answer is derived mathematically rather than guessed. RAG becomes useful later, but for a different job: once you have a large bank of generated questions, you use embeddings and metadata filters to retrieve subsets on demand — e.g. "give me 10 mirror and rotation questions." That is retrieval over structured content you already generated, not generative RAG for producing new geometry. This document lays out that full pipeline and a plan for building it into a large, labeled dataset.

2. Why Pure RAG Doesn't Work Here

A standard RAG pipeline chunks existing documents, embeds the chunks, and retrieves the closest match to a query, then asks an LLM to synthesize an answer from that context. That's a good fit when the correct answer already exists in text somewhere. It breaks down for this problem for three reasons:

- No ground truth to retrieve. A folding/punching question's answer depends on a geometric transform (reflect across fold lines, union hole positions) that isn't written down anywhere — it has to be computed.
- LLMs are unreliable at spatial transforms. Language models frequently get mirror axes, rotation direction, and hole-count reflections wrong when asked to reason over an image or a text description of one, because this is geometric computation, not language modeling.
- Chunking destroys the structure that matters. A question bank PDF chunked into text fragments loses the image-to-geometry relationship entirely — you'd retrieve question text but have no way to verify or regenerate the diagram.

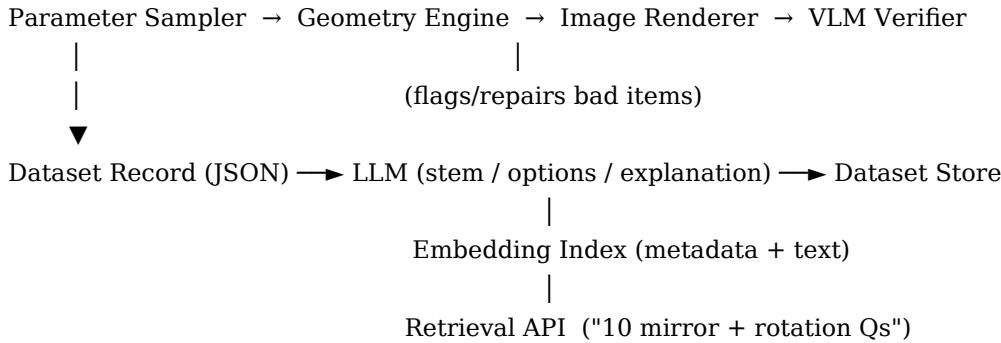
RAG-style retrieval is still valuable, just at a different stage: retrieving already-generated, already-verified questions by tag/topic/difficulty, and retrieving style examples (phrasing, distractor patterns) from real exam papers to make LLM-written question text sound authentic. Section 8 covers this.

3. Recommended Architecture

Three components, each doing the part it's actually good at:

Layer	Role	Why
Procedural geometry engine	Constructs the question and the mathematically correct answer directly from parameters (fold lines, punch points, mirror axis, rotation angle).	Guarantees correctness. No hallucination risk on the geometry itself.
Vision-language model (VLM)	Independently "solves" each rendered question as a check; also digitizes/solves scanned questions from existing books.	Catches rendering bugs and ambiguous distractors before they enter the dataset; expands the corpus beyond synthetic-only.
LLM	Writes the question stem, option text, and step-by-step explanation in natural exam language; adjusts phrasing per difficulty/exam style.	Geometry engine outputs are structured data, not prose — this is where it becomes a readable question.

Pipeline



4. Core Geometry Engine, Per Question Family

4.1 Paper Folding & Punching

Model the paper as a unit square (or rectangle) in a 2D coordinate space. Represent every fold as a line plus a stack operation:

1. Maintain a stack of active fold lines with their reflection matrices (vertical, horizontal, or diagonal at 45°).
2. Each fold reduces the visible paper region and pushes the fold's reflection transform onto the stack.
3. Punch holes are sampled as points inside the final folded region.
4. To generate the answer, unfold by popping the stack: reflect every hole across each fold line in reverse order, deduplicating overlapping points.
5. Render the folded state (question image) and the fully unfolded state with all mirrored holes (correct answer image).

Distractors are generated by deliberately corrupting one step of the correct unfold procedure: reflect across the wrong axis, skip one fold's mirroring, duplicate a hole, or offset a hole position slightly. Tag each distractor with which rule was broken — this becomes the explanation text automatically ("Option B is wrong because it omits the mirror across the second fold").

4.2 Mirror / Water Image

Take a base figure (letter, digit, clock face, arrow-and-shape composite, or an asymmetric polygon) and apply a reflection matrix across a vertical, horizontal, or specified-angle axis. Water images additionally invert vertically and can add a wavy-edge distortion for visual variety, but the transform is the same reflection primitive as folding, just without the punch/hole layer. Distractors: rotate instead of reflect, reflect across the wrong axis, or reflect only part of a composite figure.

4.3 Rotation & Series ("what's the next pattern")

Define a base figure with attributes: shape type, count, fill state, orientation, and position on a ring or grid. Define a rule that changes one or more attributes per step (rotate by a fixed angle, increment/decrement a count, cycle a fill pattern, shift position clockwise). Generate N figures in the sequence by applying the rule repeatedly, then ask for figure N+1. The correct answer is simply one more rule application. Distractors: apply the rule twice, apply it in the wrong rotational direction, or change a different attribute than the one the rule specifies.

All three families share one geometry primitive set — reflection, rotation, translation, and set operations on point/shape collections — so it's worth building them as one small transform library rather than three separate generators.

5. Unified Dataset Record Schema

Every generated (or digitized) question becomes one record, whether it came from the generator or from a scanned book. This is the schema the embedding index and retrieval API operate on:

Field	Type	Purpose
question_id	UUID	Stable identifier.
family	enum	fold_punch mirror_water rotation_series
sub_type	string	e.g. vertical_fold_double_punch, 45deg_mirror, 90deg_clockwise_series
difficulty	int 1–5	Derived from fold count / rotation steps / distractor subtlety.
params	JSON	Raw generator parameters (fold lines, punch points, axis, angle, rule) — enables regeneration or re-rendering at any resolution.
image_paths	object	Paths to the question image and each option image.
correct_option	string	Which option is correct.
distractor_rules	list	Which rule each wrong option violates — feeds the explanation text.
exam_style	enum	SSC UPSC CAT DRDO_pilot generic — controls LLM phrasing.
text	object	stem, option_labels, explanation — filled by the LLM layer.
tags	list	Free-form topic tags: mirror, vertical_axis, rotation, letters, clock_face, etc.
embedding_text	string	Concatenation of stem + tags used to build the vector index (not image pixels).
verification	object	vlm_model, vlm_answer, agree (bool), verified_at.
source	enum	synthetic scanned_book

Store images on disk/object storage and everything else as JSONL (one record per line) or a Parquet table for fast filtering at scale. Keep params for every record so any question can be re-rendered at a different resolution or style without regenerating the geometry.

6. Vision Model Verification Loop

After rendering, feed the question image (without the answer) to a vision-language model and ask it to pick the correct option and briefly explain why. Compare against the engine's known-correct answer:

- Agreement → record passes automatically into the verified dataset.
- Disagreement → flag for review; frequently indicates a rendering bug (ambiguous line overlap, holes too close to a fold line) or a distractor that's actually also defensible — both are worth fixing before the item ships.

The same VLM step lets you ingest existing question banks: scan pages from R.S. Aggarwal-style books or SSC/UPSC papers, have the vision model localize the diagram and options, solve it, and cross-check against an answer key if available. These become source: scanned_book records, expanding the dataset beyond what's procedurally generated and giving you real exam phrasing to learn style from.

Current model options (mid-2026)

Vision-language models capable enough for this kind of diagrammatic reasoning check include Qwen3-VL (dense 2B–32B and MoE 30B–235B variants, strong on spatial/document reasoning with a 256K context), GLM-4.5V and GLM-4.1V-Thinking, DeepSeek-VL for scientific/diagram-style reasoning, and Gemini 2.5 Pro as a strong proprietary option. Treat this as a swappable component behind one interface — whichever model verifies best on your held-out set of hand-checked questions is the one to use, and that ranking is worth re-checking periodically since this space moves fast.

7. LLM Layer: Turning Structured Data into a Question

The LLM never touches geometry — it only converts an already-correct structured record into natural language:

- Question stem: phrase the setup ("A square sheet of paper is folded twice as shown, then a hole is punched...") from the params.
- Option labels: render each option image and give it a letter; write any needed text captions.
- Step-by-step explanation: walk through the fold/unfold or rotation steps using distractor_rules to explain why each wrong option is wrong.
- Difficulty tuning: control via generator params first (fold count, rotation steps, subtlety of distractors); use the LLM only to adjust phrasing complexity and time-pressure framing per exam style, not to change correctness.

8. Where Retrieval (RAG) Actually Helps

This is the direct answer to "how do I make the model return 10 mirror-and-rotation questions on request": build an embedding index over embedding_text and tags for every verified record, plus a metadata filter on family/sub_type/difficulty/exam_style. A query like "give me 10 mirror and rotation questions, medium difficulty" becomes a filtered vector search — family in {mirror_water, rotation_series}, difficulty in range, ranked by embedding similarity to the query and by not having been served recently to that user. This is standard retrieval, just over your own generated-and-verified bank rather than external documents, and it also gives you deduplication for free: before adding a new synthetic question, check its embedding against the index to avoid near-duplicate geometry.

Use a vector store such as FAISS or Chroma for a local/offline setup, or pgvector if the rest of the system already lives in Postgres.

9. Scaling to a Large Dataset

Volume comes from enumerating the parameter space per family rather than hand-authoring items:

- Fold & punch: fold type (vertical/horizontal/diagonal) × fold count (1–3) × punch count (1–4) × punch position buckets on a grid. Even a modest grid resolution produces tens of thousands of distinct valid combinations per fold-count tier.
- Mirror/water: base figure library (letters, digits, composite shapes, clock faces) × axis (vertical/horizontal/diagonal) × distractor type — scales with the size of the base figure library, so invest there.
- Rotation/series: base figure attributes × rule type × sequence length (3–6 shown, 1 asked) × rotation step size.

Guardrails at scale:

- Geometric hash or embedding-similarity dedup before writing a record, so the bank doesn't fill up with near-identical items.
- Difficulty balancing: track a running histogram per family/difficulty and bias the parameter sampler toward under-represented buckets.
- Reserve a held-out slice (by parameter range, not random split) for evaluating any downstream model trained or fine-tuned on this data, so evaluation isn't contaminated by near-duplicate training items.

10. Suggested Tech Stack

Component	Suggested tools
Geometry & rendering	Python — Shapely or plain NumPy for the transform math, Pillow/OpenCV or svgwrite for rendering to PNG/SVG.
Service layer	FastAPI, exposing generate, verify, and retrieve endpoints.
Dataset storage	Images on disk/object storage (S3-compatible); records in JSONL or Parquet for fast columnar filtering.
Vector index	FAISS or Chroma (local), pgvector (if using Postgres already).
Vision verification	Qwen3-VL / GLM-4.5V / DeepSeek-VL (self-hosted) or Gemini 2.5 Pro (API) — swappable behind one interface.

Component	Suggested tools
Question text generation	Any strong instruction-tuned LLM (e.g. current Claude, GPT, or Qwen3 text models) — swappable, same reasoning.

Note on the model names in the suggestion you received: "GPT-5.5" isn't a released model as of mid-2026 — treat specific model names as a swappable config value, and re-check which VLM/LLM is currently strongest on your verification held-out set rather than hard-coding one.

11. Phased Build Plan

Phase	Deliverable
1	Fold & punch generator end-to-end: geometry engine, renderer, distractor generator, VLM verification loop, JSONL schema. Prove correctness on ~500 verified items before scaling volume.
2	Add mirror/water and rotation/series generators on the same transform library and schema; unify the parameter sampler across all three families.
3	Scale to a large volume (tens of thousands of records) with dedup and difficulty balancing; begin ingesting scanned book questions via the VLM pipeline to diversify beyond synthetic style.
4	Build the embedding index and retrieval API ("N questions of family X, difficulty Y, style Z") on top of the verified dataset; wire this into whatever front end or tutoring/exam-generation app consumes it.

12. Key Takeaway

Generation and retrieval are two different jobs here, not one. The procedural engine plus VLM verification is what makes every new question correct by construction. RAG-style embedding retrieval is what makes an already-verified bank of thousands of questions filterable and reusable on demand — that's the piece that answers "give me 10 mirror and rotation questions." Building them as separate, clearly-scoped layers (rather than asking one RAG system to do both) is what will make this reliable enough to use for something like a DRDO pilot-selection aptitude tool.